

fais une fonction qui nettoie des données de qualité de l'air

Voici une fonction Python propre et robuste pour nettoyer un jeu de données de qualité de l'air (généralement stocké dans un DataFrame pandas).

Dans le domaine de la qualité de l'air, le nettoyage implique souvent de gérer des valeurs aberrantes (comme des concentrations négatives ou anormalement élevées dues à un capteur défaillant), de convertir les dates et de boucher les trous (valeurs manquantes).

Voici le code complet avec des commentaires explicatifs :

Python



```
import pandas as pd
import numpy as np

def nettoyer_donnees_air(df, colonne_date='date', seuils_polluants=None):
    """
    Nettoie un DataFrame de données de qualité de l'air.

    Parameters:
    -----
    df : pandas.DataFrame
        Le tableau de données brut.
    colonne_date : str
        Le nom de la colonne contenant les dates/heures.
    seuils_polluants : dict
        Un dictionnaire optionnel avec les valeurs maximales réalistes pour chaque po
        Ex: {'PM2.5': 500, 'NO2': 400}

    Returns:
    -----
    pandas.DataFrame : Le DataFrame nettoyé.
    """
    # 1. Copie pour éviter de modifier le DataFrame d'origine
    df_clean = df.copy()

    # 2. Gestion des dates
    if colonne_date in df_clean.columns:
        df_clean[colonne_date] = pd.to_datetime(df_clean[colonne_date], errors='coerc
        # Supprime les lignes où la date est invalide
        df_clean = df_clean.dropna(subset=[colonne_date])
        # Trie par date et réinitialise l'index
```

```

df_clean = df_clean.sort_values(by=colonne_date).reset_index(drop=True)

# Seuils par défaut (exprimés généralement en µg/m³ ou ppm selon vos capteurs)
# Permet de repérer les anomalies flagrantes (ex: un pic à 50000 à cause d'un bug)
if seuils_polluants is None:
    seuils_polluants = {
        'PM2.5': 1000,
        'PM10': 1500,
        'NO2': 600,
        'O3': 500,
        'CO': 50 # souvent en mg/m³ ou ppm
    }

# 3. Nettoyage des polluants
for polluant, seuil_max in seuils_polluants.items():
    if polluant in df_clean.columns:
        # Conversion en numérique (force les chaînes de caractères invalides en NaN)
        df_clean[polluant] = pd.to_numeric(df_clean[polluant], errors='coerce')

        # Remplacement des valeurs aberrantes (négatives ou au-dessus du seuil) par NaN
        condition_aberrante = (df_clean[polluant] < 0) | (df_clean[polluant] > seuil_max)
        df_clean.loc[condition_aberrante, polluant] = np.nan

# 4. Imputation (remplissage) des valeurs manquantes
# Pour la qualité de l'air, l'interpolation linéaire est souvent préférable
# car la pollution est très liée au temps (la valeur à 14h ressemble à ce qu'elle était à 13h)
# limit=2 évite d'interpoler s'il manque un trop grand bloc de données d'un coup
df_clean[polluant] = df_clean[polluant].interpolate(method='linear', limit=2)

# Si des NaN restent (au début ou à la fin), on applique un "Forward Fill" puis un "Backward Fill"
df_clean[polluant] = df_clean[polluant].ffill().bfill()

return df_clean

# --- EXEMPLE D'UTILISATION ---
if __name__ == "__main__":
    # Création d'un jeu de données fictif avec des anomalies
    donnees_brutes = pd.DataFrame({
        'date': ['2026-05-01 10:00', '2026-05-01 11:00', '2026-05-01 12:00', '2026-05-01 13:00'],
        'PM2.5': [12.5, -5.0, 15.2, np.nan, 2500.0], # -5 est négatif, 2500 est hors seuil
        'NO2': [32.0, 34.5, 31.2, 29.8, 30.1]
    })

    print("--- Données brutes :")
    print(donnees_brutes)

    donnees_nettoyees = nettoyer_donnees_air(donnees_brutes)

```

```
print("\n--- Données nettoyées :")
print(donnees_nettoyees)
```

Ce que fait cette fonction en pratique :

- **Normalisation des dates** : Elle transforme les textes en vrais objets temporels manipulables par Python et trie les lignes chronologiquement.
- **Chasse aux valeurs aberrantes** : Une concentration de particules fines ou de gaz ne peut pas être négative. Les valeurs `< 0` ou supérieures aux seuils physiques (comme le `2500.0` dans l'exemple) sont automatiquement supprimées.
- **Interpolation intelligente** : Au lieu de remplacer un trou par la moyenne générale (ce qui fausserait l'analyse d'une journée ensoleillée ou d'un pic de trafic), elle trace une ligne droite entre la valeur précédente et la valeur suivante (`interpolate`).

Souhaitez-vous adapter ou pousser ce nettoyage plus loin ?

Calculer des moyennes mobiles (ex: moyenne sur 8h pour l'Ozone)



Détecter les anomalies avec le Z-Score plutôt que des seuils fixes



Écris une fonction Python qui prend une liste de mesures de qualité de l'air (dict avec city, timestamp, pm10, pm2_5...) et: (1) supprime les lignes sans timestamp, (2) remplace par None toute valeur hors des bornes 0-1000, (3) supprime les doublons (city, timestamp), (4) trie par timestamp.

Voici la fonction Python adaptée spécifiquement pour traiter une liste de dictionnaires (au format JSON / Row-by-Row) selon vos 4 règles précises.

Pour ce type de structure (sans utiliser `pandas`), on utilise les modules natifs de Python, ce qui rend le code léger et très rapide pour les API ou les fonctions Lambda.

Python



```
from datetime import datetime

def nettoyer_mesures_air(liste_mesures):
```

```
"""
```

Nettoie une liste de dictionnaires de mesures de qualité de l'air.

Règles appliquées :

1. Supprime les mesures sans 'timestamp'.
2. Remplace par None toute valeur de polluant hors des bornes [0, 1000].
3. Supprime les doublons sur le couple (city, timestamp) en gardant le premier vu
4. Trie la liste finale par 'timestamp'.

```
"""
```

```
cles_polluants = {'pm10', 'pm2_5', 'no2', 'o3', 'co'} # Liste des polluants à vé  
vus = set()  
donnees_filtrees = []
```

```
for mesure in liste_mesures:
```

```
    # (1) Suppression / Filtrage des lignes sans timestamp  
    # On vérifie que la clé existe ET qu'elle n'est pas vide/None  
    if not mesure.get('timestamp'):  
        continue
```

```
    # (3) Suppression des doublons (city, timestamp)  
    # On crée une clé unique basée sur la ville et le timestamp  
    cle_unicite = (mesure.get('city'), mesure['timestamp'])  
    if cle_unicite in vus:  
        continue # Doubleton détecté, on passe à la ligne suivante
```

```
    # Si c'est un nouvel élément, on fait une copie pour ne pas modifier l'origin  
    mesure_nettoyee = mesure.copy()
```

```
    # (2) Remplacement par None des valeurs hors bornes (0-1000)
```

```
    for cle, valeur in mesure_nettoyee.items():  
        if cle in cles_polluants and valeur is not None:  
            try:  
                valeur_num = float(valeur)  
                if not (0 <= valeur_num <= 1000):  
                    mesure_nettoyee[cle] = None  
            except (ValueError, TypeError):  
                # Si la valeur n'est pas convertible en nombre (ex: une chaîne de  
                mesure_nettoyee[cle] = None
```

```
    # On valide l'élément : on l'ajoute à notre liste et on marque la clé comme "  
    donnees_filtrees.append(mesure_nettoyee)  
    vus.add(cle_unicite)
```

```
# (4) Tri par timestamp
```

```
# Si vos timestamps sont des chaînes de caractères au format ISO (ex: "2026-05-01  
# le tri textuel fonctionne parfaitement.
```

```
donnees_filtrees.sort(key=lambda x: x['timestamp'])
```

```
return donnees_filtrees
```

```
# --- EXEMPLE D'UTILISATION ---
if __name__ == "__main__":
    mesures_brutes = [
        {'city': 'Paris', 'timestamp': '2026-07-13 12:00', 'pm10': 25, 'pm2_5': 12},
        {'city': 'Paris', 'timestamp': '', 'pm10': 30, 'pm2_5': 15},
        {'city': 'Lyon', 'timestamp': '2026-07-13 14:00', 'pm10': 1050, 'pm2_5': 18},
        {'city': 'Paris', 'timestamp': '2026-07-13 12:00', 'pm10': 99, 'pm2_5': 45},
        {'city': 'Paris', 'timestamp': '2026-07-13 11:00', 'pm10': -5, 'pm2_5': 8},
    ]

    resultat = nettoyer_mesures_air(mesures_brutes)

    import json
    print(json.dumps(resultat, indent=2))
```

Quelques détails sur les choix techniques :

- **Performance du dédoublement** : L'utilisation d'un `set()` pour stocker la `cle_unicite` permet une vérification instantanée en temps constant $O(1)$, ce qui évite de ralentir le code même si votre liste contient des milliers de lignes.
- **Tolérance aux erreurs** : Le bloc `try/except` à l'étape (2) permet d'anticiper le cas où une valeur de polluant arriverait sous forme de texte non convertible (ex: `'Erreur capteur'`), en la remplaçant proprement par `None` plutôt que de faire crasher votre script.

write a function to clean air quality data

Here is a complete, robust Python function using the **pandas** library, which is the industry standard for handling data pipelines.

In air quality monitoring, data cleaning typically requires handling physical anomalies (like negative concentrations or impossible electrical spikes), fixing timestamps, and managing missing values (gaps in data).

Python



```
import pandas as pd
import numpy as np
```

```

def clean_air_quality_data(df, datetime_col='timestamp', pollutant_thresholds=None):
    """
    Cleans an air quality DataFrame by handling dates, filtering physical outliers,
    removing duplicates, and intelligently interpolating missing data.

    Parameters:
    -----
    df : pandas.DataFrame
        The raw input dataframe.
    datetime_col : str
        The name of the column containing dates/timestamps.
    pollutant_thresholds : dict, optional
        Custom max thresholds for pollutants. If None, safe defaults are used.

    Returns:
    -----
    pandas.DataFrame : The cleaned, sorted, and filled dataframe.
    """
    # 1. Create a copy to prevent SettingWithCopyWarning or modifying the original data
    df_clean = df.copy()

    # 2. Datetime Standardisation & Sorting
    if datetime_col in df_clean.columns:
        df_clean[datetime_col] = pd.to_datetime(df_clean[datetime_col], errors='coerce')
        # Drop rows where the timestamp is completely broken
        df_clean = df_clean.dropna(subset=[datetime_col])
        # Chronological sort is vital for correct time-series interpolation later
        df_clean = df_clean.sort_values(by=datetime_col).reset_index(drop=True)

    # 3. Deduplication (e.g., if multiple sensors or retries sent data for the same sensor)
    subset_cols = [datetime_col]
    if 'city' in df_clean.columns:
        subset_cols.append('city')
    elif 'sensor_id' in df_clean.columns:
        subset_cols.append('sensor_id')

    df_clean = df_clean.drop_duplicates(subset=subset_cols, keep='first')

    # Standard realistic upper limits (in µg/m³ or ppm) to catch sensor glitched spikes
    if pollutant_thresholds is None:
        pollutant_thresholds = {
            'pm2_5': 1000,
            'pm10': 1500,
            'no2': 600,
            'o3': 500,
            'co': 50
        }

    # 4. Outlier Handling & Missing Data Imputation

```

```

for pollutant, max_limit in pollutant_thresholds.items():
    if pollutant in df_clean.columns:
        # Force everything to numeric, turning text errors (like "Sensor Error")
        df_clean[pollutant] = pd.to_numeric(df_clean[pollutant], errors='coerce')

        # Replace physical impossibilities (< 0 or > threshold) with NaN
        is_outlier = (df_clean[pollutant] < 0) | (df_clean[pollutant] > max_limit)
        df_clean.loc[is_outlier, pollutant] = np.nan

        # Linear interpolation is usually the most accurate way to fill short gap
        # in environmental data because air quality changes gradually over time.
        # limit=2 ensures we don't accidentally fill massive hours-long blackouts
        df_clean[pollutant] = df_clean[pollutant].interpolate(method='linear', li

        # Fallback for remaining edge NaNs at the very beginning or end of the da
        df_clean[pollutant] = df_clean[pollutant].ffill().bfill()

return df_clean

# --- QUICK DEMO ---
if __name__ == "__main__":
    raw_data = pd.DataFrame({
        'timestamp': ['2026-07-13 10:00', '2026-07-13 12:00', '2026-07-13 11:00', '20
        'pm2_5': [12.5, -4.0, 14.2, np.nan, 2500.0], # Includes a negative, a missing
        'no2': [32.0, 31.2, 34.5, 29.8, 29.8]
    })

    print("Raw Data:")
    print(raw_data)

    print("\nCleaned Data:")
    print(clean_air_quality_data(raw_data))

```

Why this approach works well:

- **Time-Series Aware:** By sorting chronologically first, the `.interpolate()` method calculates real linear trends between the previous hour and the next hour, rather than just filling gaps with a generic dataset mean.
- **Failsafe Enforcements:** Converting with `errors='coerce'` handles unexpected raw data anomalies (like empty spaces or text string warnings) seamlessly without breaking your pipeline execution.

Write a Python function that takes a list of air quality measurements (dict with city, timestamp, pm10, pm2_5...)

and: (1) drops rows with missing timestamp, (2) replaces out-of-range values (0-1000) with None, (3) removes duplicates on (city, timestamp), (4) sorts by timestamp.

Here is the clean, efficient Python function using native data structures (list of dictionaries) to apply your four rules precisely:

Python



```
def clean_air_quality_records(measurements):  
    """  
    Cleans a list of air quality measurement dictionaries.  
  
    Rules applied:  
    1. Drops rows missing a 'timestamp'.  
    2. Replaces values outside the [0, 1000] range with None for pollutants.  
    3. Removes duplicates on (city, timestamp), keeping the first seen.  
    4. Sorts the final list chronologically by timestamp.  
    """  
    pollutant_keys = {'pm10', 'pm2_5', 'no2', 'o3', 'co'}  
    seen_records = set()  
    cleaned_list = []  
  
    for record in measurements:  
        # (1) Drop rows with missing timestamp  
        # Safeguards against missing key or falsy values like None or ""  
        if not record.get('timestamp'):  
            continue  
  
        # (3) Deduplicate based on (city, timestamp)  
        unique_key = (record.get('city'), record['timestamp'])  
        if unique_key in seen_records:  
            continue  
  
        # Shallow copy to avoid mutating the original input list elements  
        cleaned_record = record.copy()  
  
        # (2) Replace out-of-range values (0-1000) with None  
        for key, value in cleaned_record.items():  
            if key in pollutant_keys and value is not None:  
                try:  
                    num_value = float(value)  
                    if not (0 <= num_value <= 1000):  
                        cleaned_record[key] = None  
                except (ValueError, TypeError):
```



```

        # Handle cases where value is a non-numeric string (e.g., "Error")
        cleaned_record[key] = None

    cleaned_list.append(cleaned_record)
    seen_records.add(unique_key)

# (4) Sort by timestamp
# ISO-8601 strings (e.g., "2026-07-13 12:00") sort perfectly via natural string s
cleaned_list.sort(key=lambda x: x['timestamp'])

return cleaned_list

# --- QUICK VERIFICATION ---
if __name__ == "__main__":
    raw_data = [
        {'city': 'London', 'timestamp': '2026-07-13 12:00', 'pm10': 22, 'pm2_5': 11},
        {'city': 'London', 'timestamp': '', 'pm10': 30, 'pm2_5': 15},
        {'city': 'Paris', 'timestamp': '2026-07-13 14:00', 'pm10': 1250, 'pm2_5': 18},
        {'city': 'London', 'timestamp': '2026-07-13 12:00', 'pm10': 95, 'pm2_5': 40},
        {'city': 'London', 'timestamp': '2026-07-13 11:00', 'pm10': -8, 'pm2_5': 7},
    ]

    import json
    print(json.dumps(clean_air_quality_records(raw_data), indent=2))

```

Design Notes:

- **Time Complexity Performance:** Using a Python `set()` for unique tracking handles deduplication in $O(1)$ constant time lookup per row. The script will remain fast even on arrays of tens of thousands of measurements.
- **Robust Range Checks:** The `try/except` wrapper ensures that if a sensor accidentally drops a string status code into a pollutant value field (e.g., `'FAULT'`), the function will smoothly convert it to `None` instead of crashing.